COMSATS University Islamabad, Lahore Campus

CIIT, Defence Road, Off Raiwind Road, Lahore

Assignment # 2
Due date 25-10-2026

CLO 1: Employ linear data structures to solve computing problems. (Applying)

Problem Statement:

Recall the concepts of stacks. Implement a stack using arrays. Build a program that takes an infix statement and converts it to a postfix statement. If the statement is incorrect the program should inform the user that the input statement is incorrect. Otherwise, it should return the postfix expression.

After the statement has been successfully converted, the program should then evaluate this postfix expression and return the result.

(Infix to Postfix) Write a program that converts an infix expression into an equivalent postfix expression.

The rules to convert an infix expression into an equivalent postfix expression are as follows:

Suppose `infx` represents the infix expression and `pfx` represents the postfix expression. The rules to convert `infx` into `pfx` are as follows:

a. Initialize `pfx` to an empty expression and also initialize the stack.

b. Get the next symbol, `sym`, from `infx`.

   b.1. If `sym` is an operand, append `sym` to `pfx`.

   b.2. If `sym` is `(`, push `sym` into the stack.

   b.3. If `sym` is `)`, pop and append all the symbols from the stack until the most recent left parenthesis. Pop and discard the left parenthesis.

   b.4. If `sym` is an operator:

      b.4.1. Pop and append all the operators from the stack to `pfx` that are above the most recent left parenthesis and have precedence greater than or equal to `sym`.

      b.4.2. Push `sym` onto the stack.

c. After processing `infx`, some operators might be left in the stack. Pop and append to `pfx` everything from the stack.

In this program, you will consider the following (binary) arithmetic operators: `+`, `-`, `*`, and `/`. You may assume that the expressions you will process are error free.

Modify above program to check an expression for balanced brackets. For example,
Expression: [x+{y*z+(a+c)}] It has bracket sequence: [{()}] which is balanced
Expression: [x+{y*z+(a+c})] It has bracket sequence: [{()}] which are not balanced